

Graph Problem Solving — General Framework

Har graph problem pe apply karne wala 5-step universal checklist

Ye ek **general template** hai jo tum **har graph problem** pe apply kar sakte ho — chahe woh path ho, cycle ho, MST ho, ya kuch bhi. Iska maksad hai statement ke literal wording mein phasne se bachna aur asli underlying structure pehchanna.

STEP 1: Statement se 'Goal' identify karo

Pehle dekho actually poocha kya gaya hai (surface level):

- Path?
- Shortest path?
- Connected component?
- Cycle detection?
- Number of ways?
- MST?
- Bipartite check?

■ **Trap:** Keywords se turant pattern-match mat karo. "Path" sunke shortest-path algorithms (Dijkstra/BFS) mat kood jao. Pehle poora statement padho, especially constraints wali lines — wahi asli meaning decide karti hain.

STEP 2: Path/Traversal ke 'Restrictions' nikaalo

Ye **sabse critical step** hai — yahin log galti karte hain. Har baar khud se poocho:

Question	Kyun important?
Node revisit allowed hai?	Simple path vs walk
Edge revisit allowed hai?	Eulerian vs normal path
Simple path chahiye (no repeats)?	DFS with visited-set
Directed hai ya Undirected?	Traversal direction, cycle logic
Weighted hai?	Dijkstra/Bellman-Ford vs plain BFS
Negative weights?	Bellman-Ford vs Dijkstra

Ye info usually ek **innocent-looking line** mein chhupi hoti hai jise log skip kar dete hain (jaise: "same road/city may be visited more than once").

Rule of thumb: Agar statement mein "simple path" explicitly nahi likha, to assume mat karo ki nodes/edges unique honge.

STEP 3: Restriction ka 'Effect' socho (chhote example pe)

Ek chhota graph (4-5 nodes) mentally ya paper pe banao aur restriction apply karke dekho:

- Agar revisit allowed hai → kya tum poore connected component mein ghoom sakte ho?
- Agar simple path chahiye → kya options limited ho jaate hain?
- Agar cycle allowed hai → kya problem "component-level" ban jaati hai?

Example graph:

```

1 ----- 2
|         |
|         |
|         |
3 ----- 4

```

Agar revisit allowed hai to ye path bhi VALID hai:

```
1 -> 2 -> 4 -> 2 -> 1 -> 3 -> 4
```

Isse tumhe **actual search space** samajh aata hai — jo statement ke literal wording se bahut different ho sakta hai. Yahan path ka concept practically "connected component mein kahin bhi ghoomna" ban gaya.

STEP 4: 'Answer kis cheez par depend karta hai?' — yahi asli insight hai

Khud se poocho: Agar main kahin bhi ghoom sakta hoon (ya jo bhi restriction hai), to final answer **specific path** par depend karta hai ya kisi **broader structure** (component, set of nodes, set of edges) par?

Common transformations:

- Revisit allowed + connected → answer = **connected component ka property** (min/max/sum of all elements)
- Simple path required → answer = **actual path search** (DFS/backtracking)
- "Number of ways" → answer = **counting via DFS/DP**
- Minimum connections to make graph connected → **Union-Find**

Yahi step problem ko **reframe** karta hai — literal statement se hatke uska **effective/equivalent form** dhoondhta hai.

STEP 5: Reframed problem ke hisaab se Algorithm choose karo

Reframed Problem	Algorithm
Connected component traversal/property	DFS / BFS / Union-Find
Shortest path (unweighted)	BFS
Shortest path (weighted, no negative)	Dijkstra
Shortest path (negative weights)	Bellman-Ford
All-pairs shortest path	Floyd-Warshall
Cycle detection (undirected)	DFS / Union-Find
Cycle detection (directed)	DFS with recursion stack / Kahn's (topo sort)
MST	Kruskal / Prim

Counting paths	DFS + memoization (DP on graph)
Bipartite check	BFS/DFS with 2-coloring

QUICK RECAP — 5 Questions Template

1. **Statement se kya pooch rahe hain?** (surface goal)
2. **Restrictions kya hain?** (revisit? directed? weighted?)
3. **Restriction ka effect kya hai?** (chhota example try karo)
4. **Answer kis par depend karta hai?** (path vs component vs set)
5. **Konsa algorithm fit baithta hai?** (reframed problem ke hisaab se)

Isse tum kisi bhi naye graph problem ko systematically break kar sakte ho, bina keyword-trap mein phanse.